POLITECNICO MILANO 1863 | DIPARTIMENTO DI ELETTRONICA INFORMAZIONE E BIOINGEGNERIA

# Laboratorio 2
# OpenFlow – Regole Proattive

11.03.2026 I Software Defined Networking

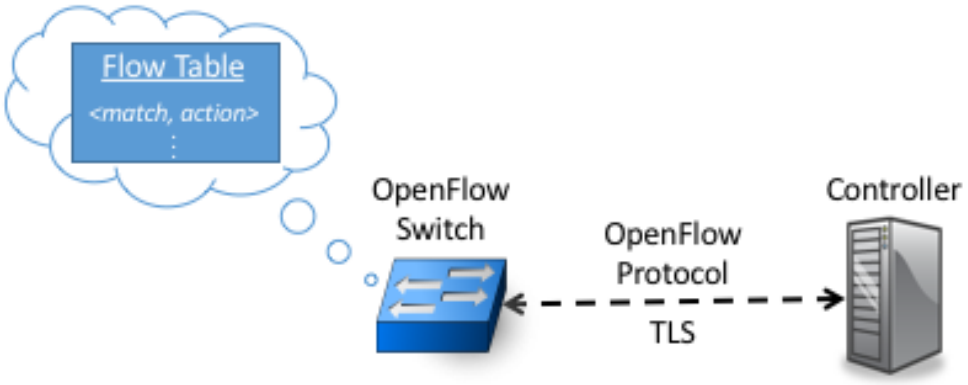
# OpenFlow

1

# Recap

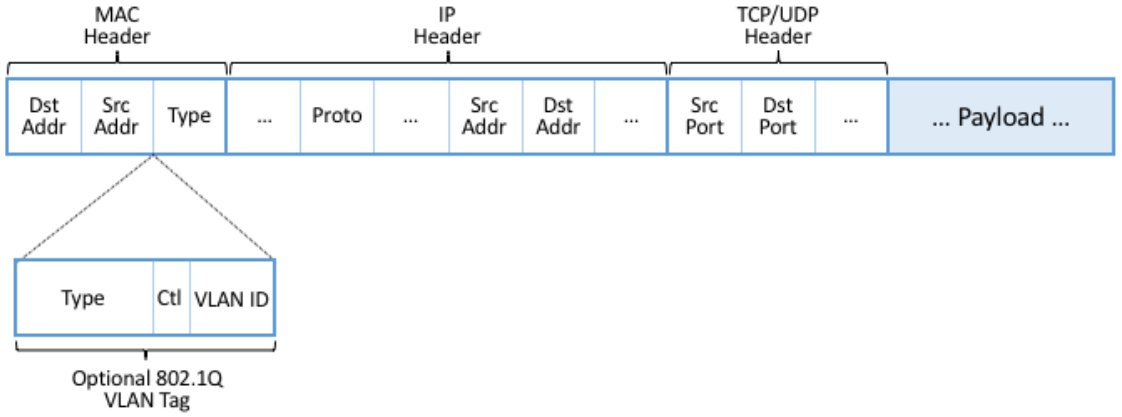
1.1

# Prima generazione: la programmabilità del piano di controllo




Il piano dati segue l'astrazione «Flow Table»:

- Regole di match sui campi dei pacchetti
- Azioni da una lista (inoltra, modifica, scarta, …)

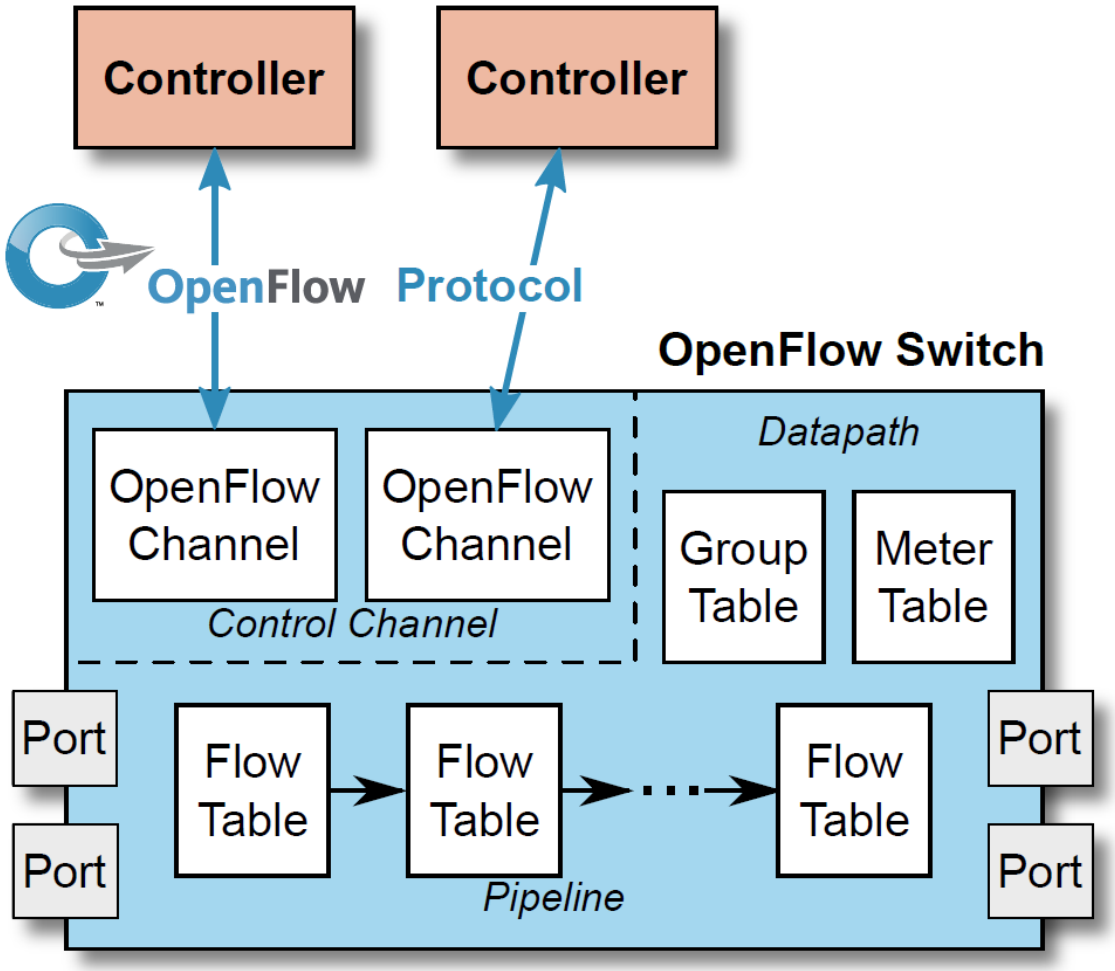


# OpenFlow

Lo standard OpenFlow definisce:

- Un modello astratto di switch
- Le azioni che il controller può compiere sullo switch
- Il protocollo tra switch e controllore

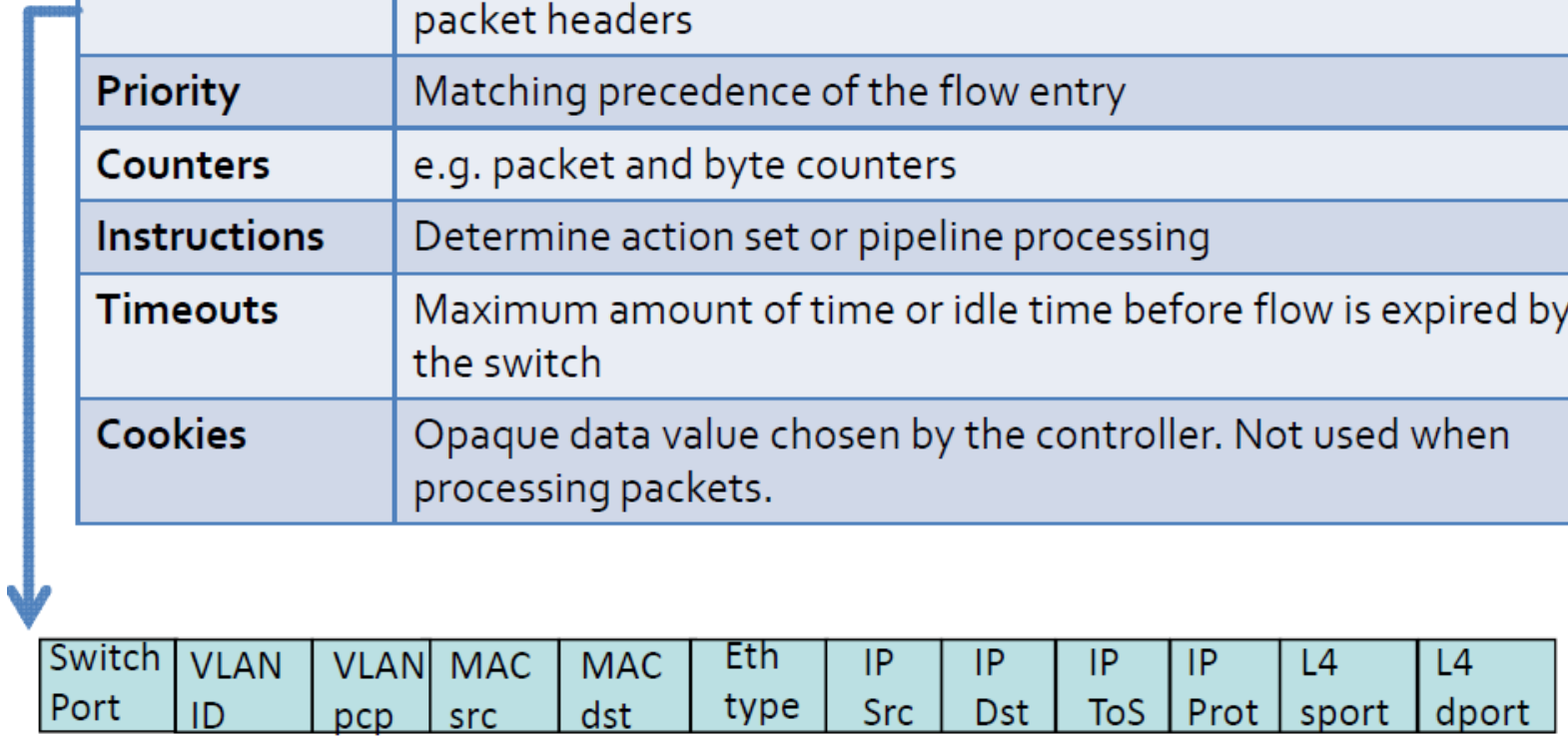
OpenFlow non è uno standard IETF
Usiamo la versione 1.3 (Aprile 2012)



# Flow Table Entries

**Main components of a flow entry in a flow table.**

| | |
|---|---|
| **Match fields** | To match against packets. These consist of the ingress port and packet headers |
| **Priority** | Matching precedence of the flow entry |
| **Counters** | e.g. packet and byte counters |
| **Instructions** | Determine action set or pipeline processing |
| **Timeouts** | Maximum amount of time or idle time before flow is expired by the switch |
| **Cookies** | Opaque data value chosen by the controller. Not used when processing packets. |




The match field contains either a specific value or a “wildcard”

# Regole

- La ricerca è esatta su tutti i campi
- Un campo può essere non specificato (wildcard)
- La ricerca restituisce solo la flow entry a massima priorità
- Flow entry potenzialmente sovrapposte e con la stessa priorità sono rifiutate dallo switch
- La regola di default («table-miss entry») ha una wildcard su tutti i campi e priorità zero
- Se non ci sono corrispondenze e la table-miss entry non è presente, allora il pacchetto è scartato

# Esempi

Switching

| Switch Port | MAC src | MAC dst | Eth type | VLAN ID | IP Src | IP Dst | IP Prot | TCP sport | TCP dport | Action |
|---|---|---|---|---|---|---|---|---|---|---|
| * | * | 00:1f:.. | * | * | * | * | * | * | * | port6 |

Flow Switching

| Switch Port | MAC src | MAC dst | Eth type | VLAN ID | IP Src | IP Dst | IP Prot | TCP sport | TCP dport | Action |
|---|---|---|---|---|---|---|---|---|---|---|
| port3 | 00:20.. | 00:1f.. | 0800 | vlan1 | 1.2.3.4 | 5.6.7.8 | 4 | 17264 | 80 | port6 |

Firewall

| Switch Port | MAC src | MAC dst | Eth type | VLAN ID | IP Src | IP Dst | IP Prot | TCP sport | TCP dport | Action |
|---|---|---|---|---|---|---|---|---|---|---|
| * | * | * | * | * | * | * | * | * | 22 | drop |

# Caratteristiche

1.2

# Porte OpenFlow

- Rappresentano le interfacce di entrata e di uscita dallo switch
- Interfacce fisiche (1,2,3,…)
- Interfacce riservate:
  - ALL/FLOOD: tutte le porte tranne la porta di ingresso del pacchetto
  - IN_PORT: la porta di ingresso del pacchetto
  - CONTROLLER:
    - Il pacchetto è arrivato dal controllore incapsulato in un messaggio *Packet-Out*
    - Il pacchetto va spedito al controllore incapsulato in un messaggio *Packet-In*

# Istruzioni (alcune)

- Apply-Actions <lista di azioni>:
  - esegue le azioni immediatamente sul pacchetto

- Goto-Table <id tabella>:
  - Invia il pacchetto alla tabella specificata
  - Per evitare loop, l'id della nuova tabella deve essere maggiore dell'id della tabella corrente

# Azioni (alcune)

- Output ‹porta›: invia il pacchetto a ‹porta›

Non esiste un'azione esplicita di drop: l'assenza di una istruzione Output implica che il pacchetto è scartato.

- Set Field ‹campo›, ‹valore›: sovrascrive un campo del pacchetto
- Set IP TTL ‹nuovo TTL›
- Decrement IP TTL

# Modelli di controllo: Reattivo vs Proattivo

| MODELLO PROATTIVO | MODELLO REATTIVO |
| --- | --- |
| • All'accensione lo switch chiede le regole al controllore<br>• Il controllore installa tutte le regole | • Flow table inizialmente vuota<br>• Al primo pacchetto di un nuovo flusso si chiede al controllore («Packet-In»)<br>• Il controllore installa una regola per il nuovo flusso |

# Modelli di controllo: Reattivo vs Proattivo

| MODELLO PROATTIVO | MODELLO REATTIVO |
| --- | --- |
| • Zero flow setup time<br>• La perdita di connessione non causa malfunzionamenti<br>• La tabella di forwarding potrebbe contenere delle regole che non sono necessarie | • Uso efficiente della tabella di forwarding -- solo le regole necessarie sono installate<br>• Il traffico potrebbe subire dei ritardi, visto che dobbiamo aspettare che il controller installi le regole<br>• Se la connessione con il controller è persa, lo switch potrebbe non funzionare correttamente |

# Ryu

2

# Funzionamento Ryu

2.1

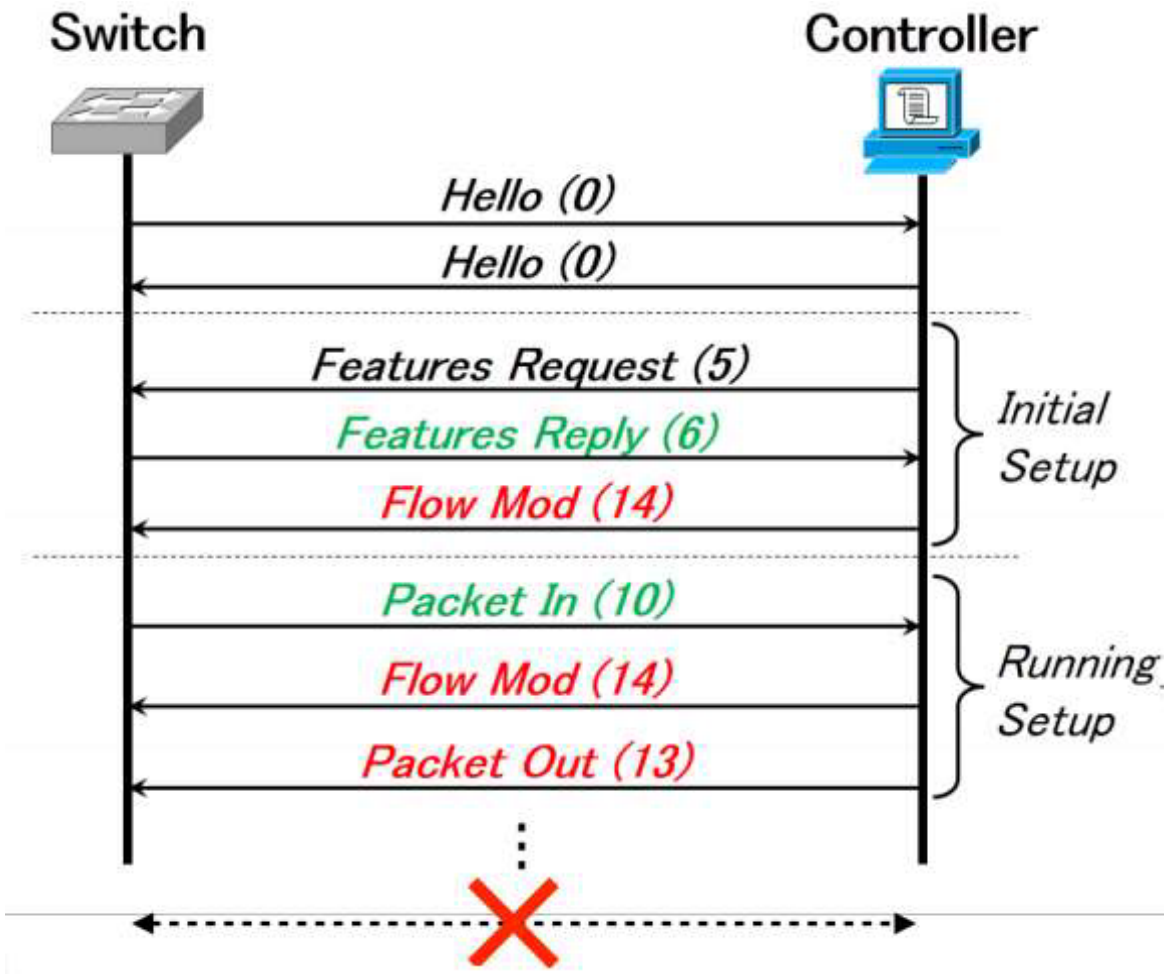
# Esercizio

Ryu è un controller OpenFlow programmabile in *Python*.
L'interfaccia northbound è molto limitata. Principalmente espone una API Python basata sull'interfaccia southbound.

La programmazione in Ryu consiste nello scrivere funzioni che vengono invocate in corrispondenza di determinati eventi:

- Arriva una Packet-In
- Arriva la risposta a una richiesta del controllore
- …

# Stati sella comunicazione




HANDSHAKE_DISPATCHER

CONFIG_DISPATCHER

MAIN_DISPATCHER

DEAD_DISPATCHER

# Let’s get our hands dirty!

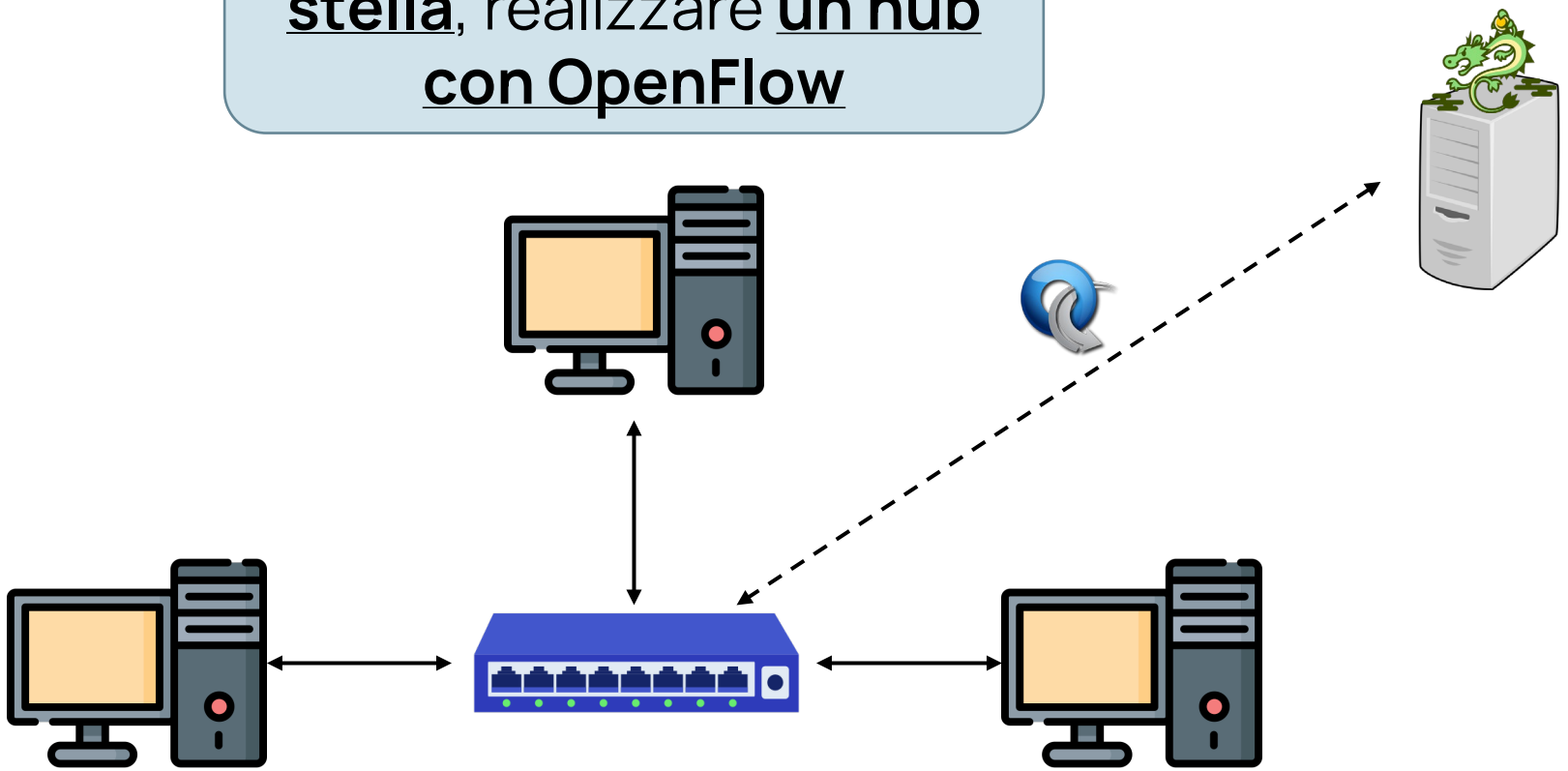


# Hands-on (guidato)

2.2

# Esercizio 1

Data una topologia **a stella**, realizzare **un hub con OpenFlow**

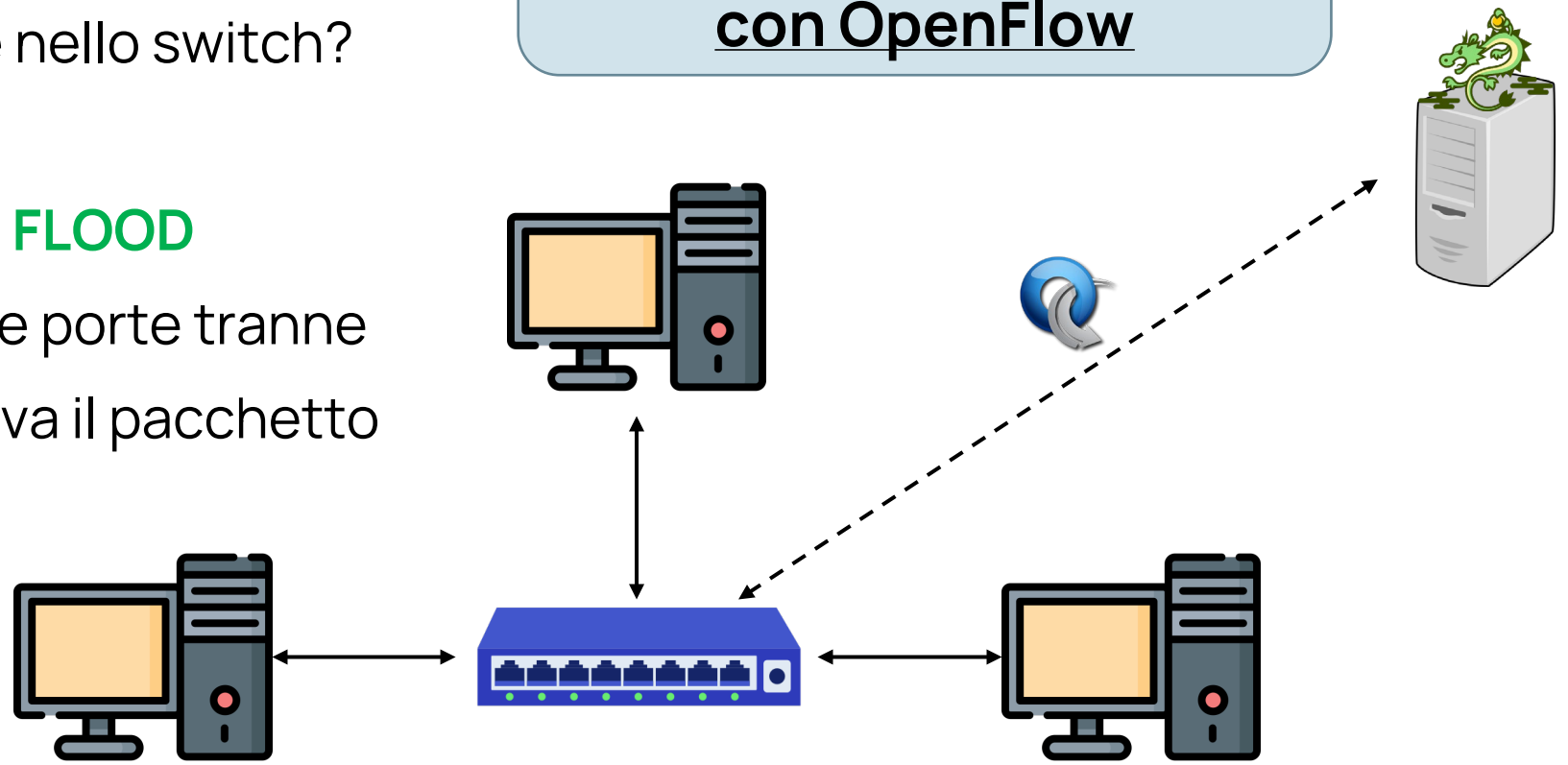


# Esercizio 1

Data una topologia **a stella**, realizzare **un hub con OpenFlow**

Quale regola devo inserire nello switch?

- **MATCH -> ANY**
- **Action: OUTPUT -> FLOOD**
  - Inoltro su tutte le porte tranne quella da cui arriva il pacchetto

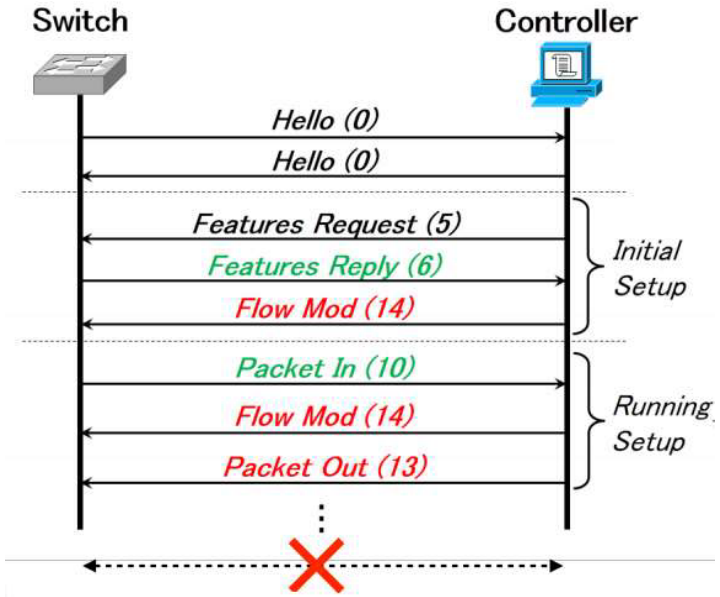


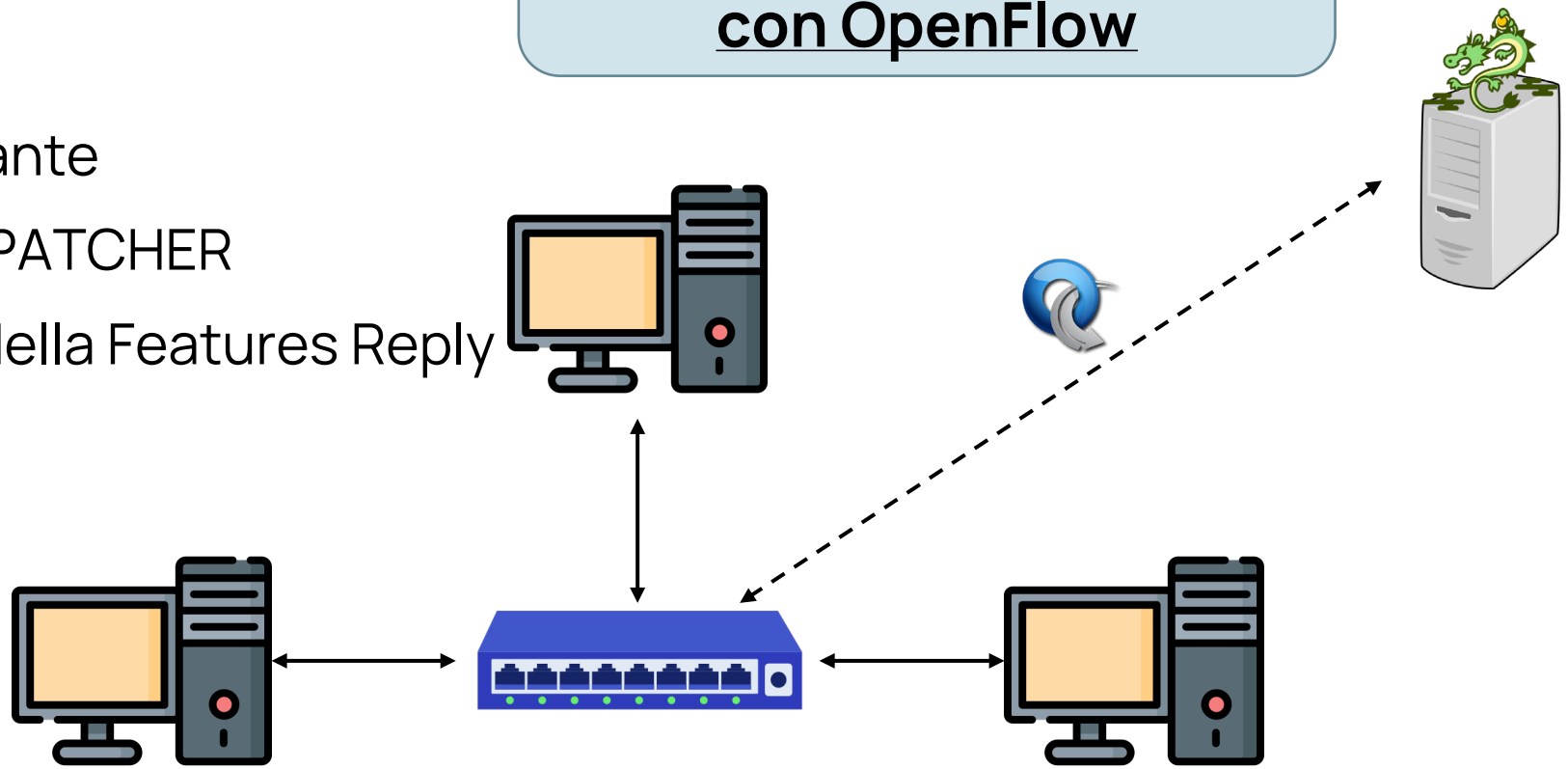
# Esercizio 1

Data una topologia **a stella**, realizzare **un hub con OpenFlow**

Quando devo inserire la regola?

- Approccio PROATTIVO
- Appena possibile, durante la fase di CONFIG_DISPATCHER
  - Dopo la ricezione della Features Reply



# Programma base

```
from ryu.base import app_manager
from ryu.controller import ofp_event
from ryu.controller.handler import set_ev_cls, CONFIG_DISPATCHER
from ryu.ofproto import ofproto_v1_3

class PolimiHub(app_manager.RyuApp):
        OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

        @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
        def switch_features_handler(self, ev):
                datapath = ev.msg.datapath
                ofproto = datapath.ofproto
                parser = datapath.ofproto_parser
                match = parser.OFPMatch()
                actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
                inst = [ parser.OFPInstructionActions(
                        ofproto.OFPIT_APPLY_ACTIONS, actions ) ]
                mod = parser.OFPFlowMod( datapath=datapath, priority=1,
                        match=match, instructions=inst )
                datapath.send_msg(mod)
```

# Sezione import

```
from ryu.base import app_manager
from ryu.controller import ofp_event
from ryu.controller.handler import set_ev_cls, CONFIG_DISPATCHER
from ryu.ofproto import ofproto_v1_3
```

Importiamo tra l'altro:

- Gli eventi della fase di CONFIG_DISPATCHER
- Le procedure di OpenFlow 1.3

# Applicazione Ryu

```
class PolimiHub(app_manager.RyuApp):
        OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
```




Per questa applicazione non ci servono variabili di class/istanza, dobbiamo solo dichiarare quali versioni di OpenFlow vogliamo gestire

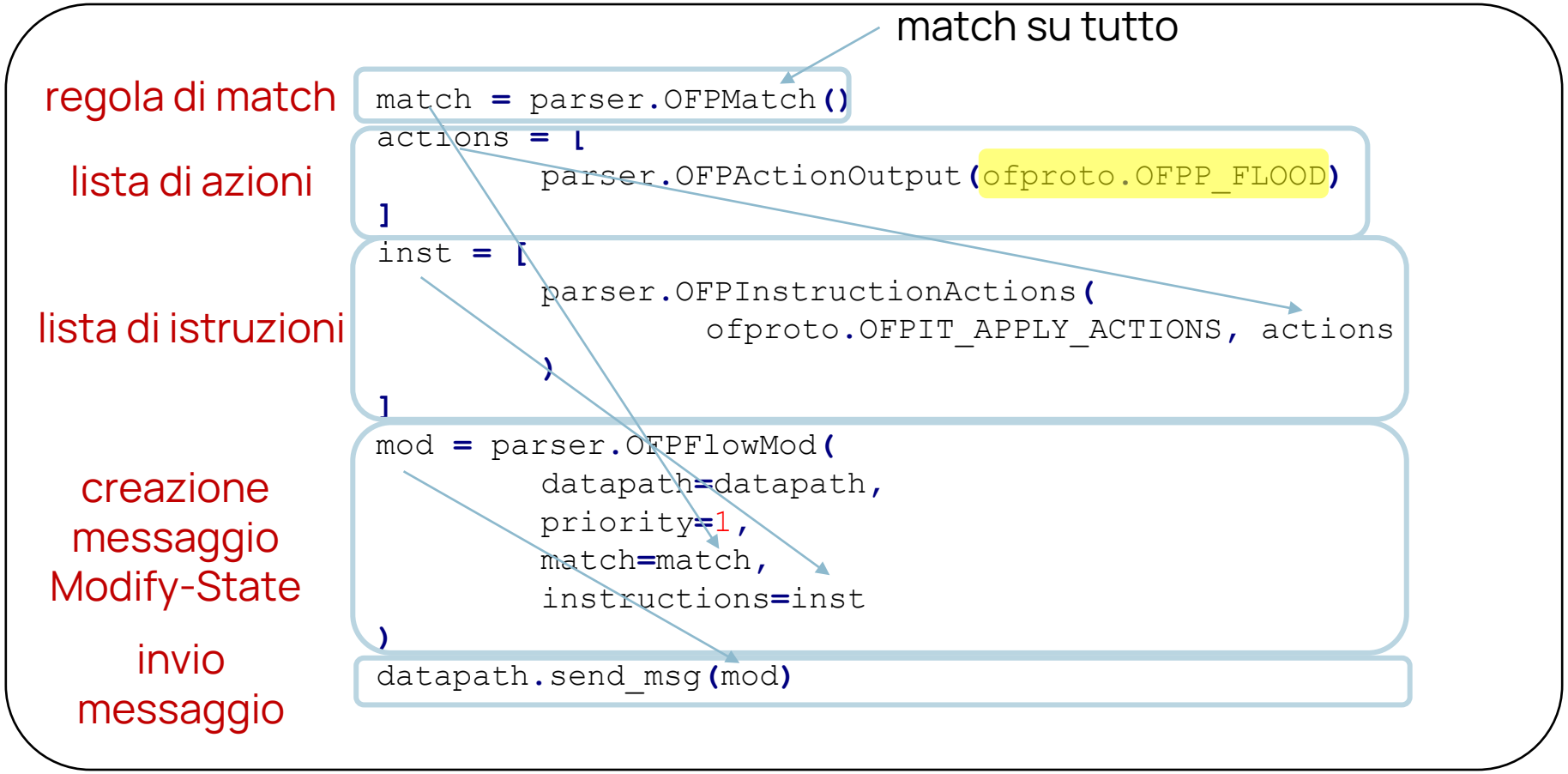
# Event Handler

```
@set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
def switch_features_handler(self, ev):
        datapath = ev.msg.datapath
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser
```

Messaggi verso lo switch

Il gestore dell'evento *EventOFPSwitchFeatures* viene chiamato quando arriva un messaggio FeaturesReply. Tale messaggio contiene i parametri del datapath (ovvero dello switch)

# Creazione e invio della regola



# Esecuzione esercizio 1

Esecuzione del test:

$ sudo mn –mac --topo single,3 --controller remote

$ ryu-manager sdn-labs/2_hub/hub1.py flowmanager/flowmanager.py

Possiamo aggiungere flowmanager per vedere le regole installate:

http://localhost:8080/home/index.html

# Risultato esercizio 1

Regola installata:

| | PRIORITY | MATCH FIELDS | COOKIE | DURATION | IDLE TIMEOUT | HARD TIMEOUT | INSTRUCTIONS | PACKET COUNT | BYTE COUNT | FLAGS |
|---|---|---|---|---|---|---|---|---|---|---|
| ☐ | 1 | ANY | 0 | 33 | 0 | 0 | OUTPUT:FLOOD | 24 | 1680 | 0 |

Testiamo la raggiungibilità:
mininet› pingall
…0% drop…

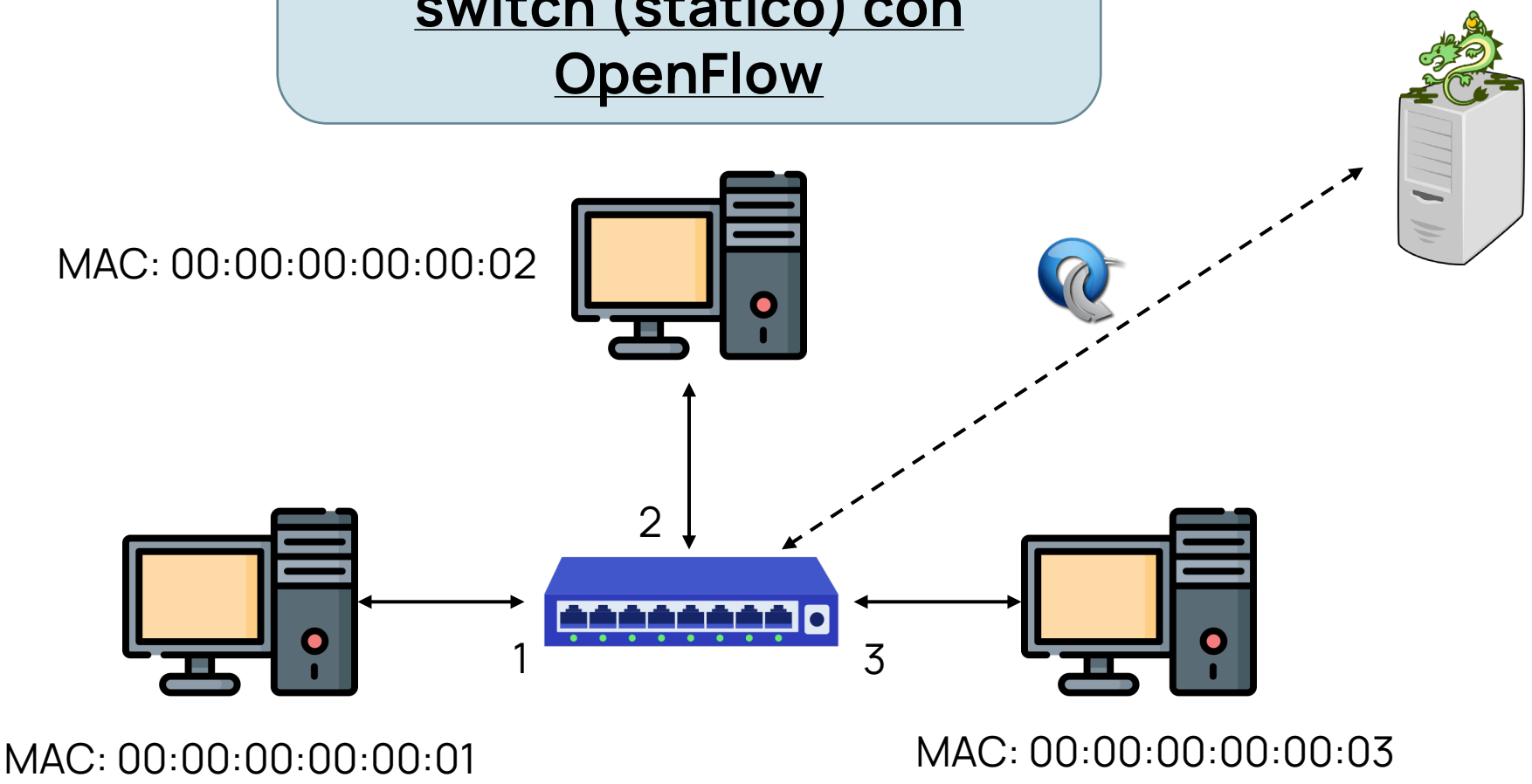


# Hands-on

2.3

# Esercizio 2

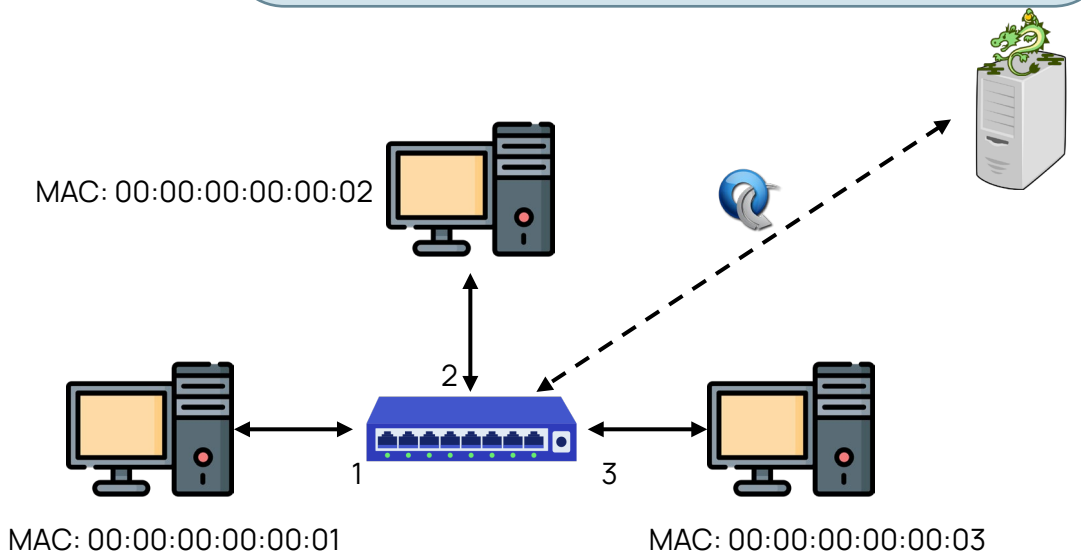
Data una topologia **a stella con 3 host**, realizzare **uno switch (statico) con OpenFlow**



# Esercizio 2 - hints

1. Trucco: con mininet possiamo assegnare MAC progressivi agli host (altrimenti dobbiamo scoprirli)
   - Basta utilizzare --mac
2. È possibile debuggare la rete aprendo una sessione tcpdump su un host per vedere cosa succede
   - mininet/util/m h2 tcpdump –enl
3. In alternativa, si può usare:
   - xterm h2: apre un nuovo terminale su h3 su cui eseguire i comandi -> tcpdump -enl

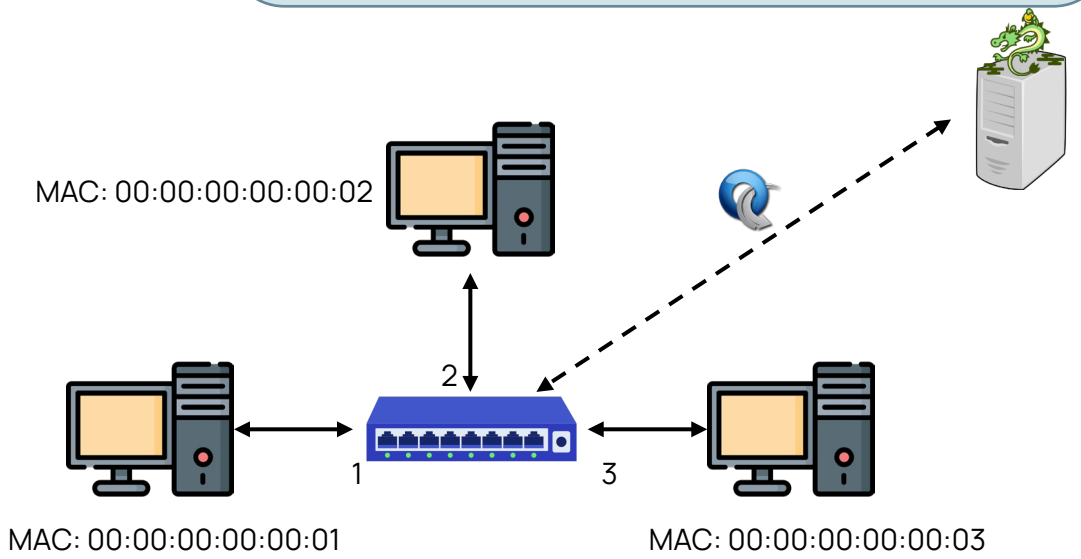
Data una topologia **a stella con 3 host**, realizzare **uno switch (statico) con OpenFlow**



# Esercizio 2 - hints

Data una topologia **a stella con 3 host**, realizzare **uno switch (statico) con OpenFlow**

4. Dobbiamo insegnare le seguenti flow rules:
   - Default -> inviare a tutti
   - MAC destinazione '00:00:00:00:00:01' -> inviare a porta 1
   - MAC destinazione '00:00:00:00:00:02' -> inviare a porta 2
   - MAC destinazione '00:00:00:00:00:03' -> inviare a porta 3




Esempio di regola di match:

```
match=parser.OFPMatch(eth_dst='00:00:00:00:00:01')
```

# Tabella completa

SW_1

Flow Table 0

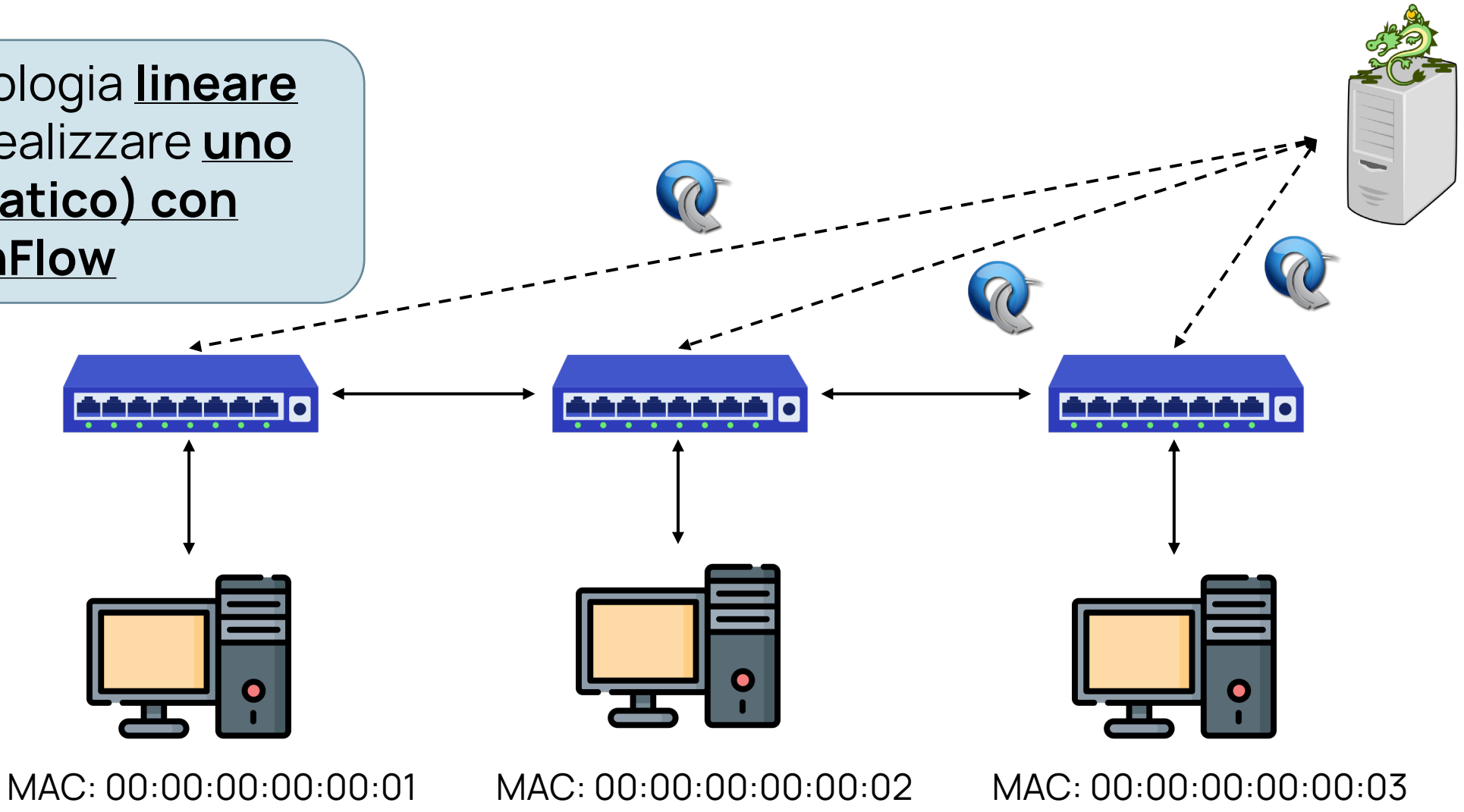
| PRIORITY | MATCH FIELDS | COOKIE | DURATION | IDLE TIMEOUT | HARD TIMEOUT | INSTRUCTIONS | PACKET COUNT | BYTE COUNT | FLAGS |
|---|---|---|---|---|---|---|---|---|---|
| 1 | eth_dst = ff:ff:ff:ff:ff:ff | 0 | 709 | 0 | 0 | OUTPUT:FLOOD | 56 | 2352 | 0 |
| 1 | eth_dst = 00:00:00:00:00:01 | 0 | 205 | 0 | 0 | OUTPUT:1 | 17 | 1162 | 0 |
| 1 | eth_dst = 00:00:00:00:00:02 | 0 | 126 | 0 | 0 | OUTPUT:2 | 13 | 938 | 0 |
| 1 | eth_dst = 00:00:00:00:00:03 | 0 | 81 | 0 | 0 | OUTPUT:3 | 7 | 518 | 0 |

# Hands-on - Lineare

2.4

# Esercizio 3

Data una topologia **lineare con 3 host**, realizzare **uno switch (statico) con OpenFlow**



# Esercizio 3 - hints

Differenziare gli switch con:

if (datapath.id == ‹numero switch›):

Data una topologia **lineare con 3 host**, realizzare **uno switch (statico) con OpenFlow**

